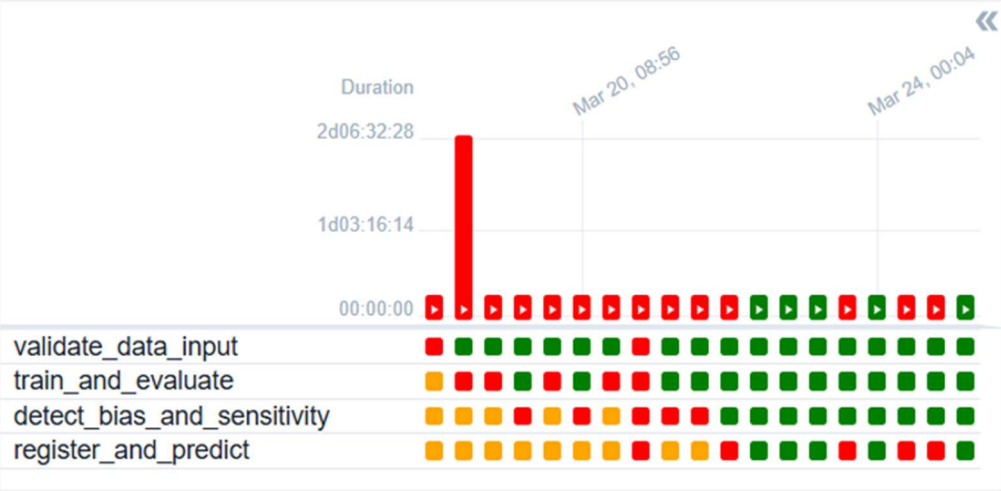
BlueBikes Demand Forecasting

Model Pipeline : MLOps End-to-End Overview

Date: ____03/24/2026____



1. Overview

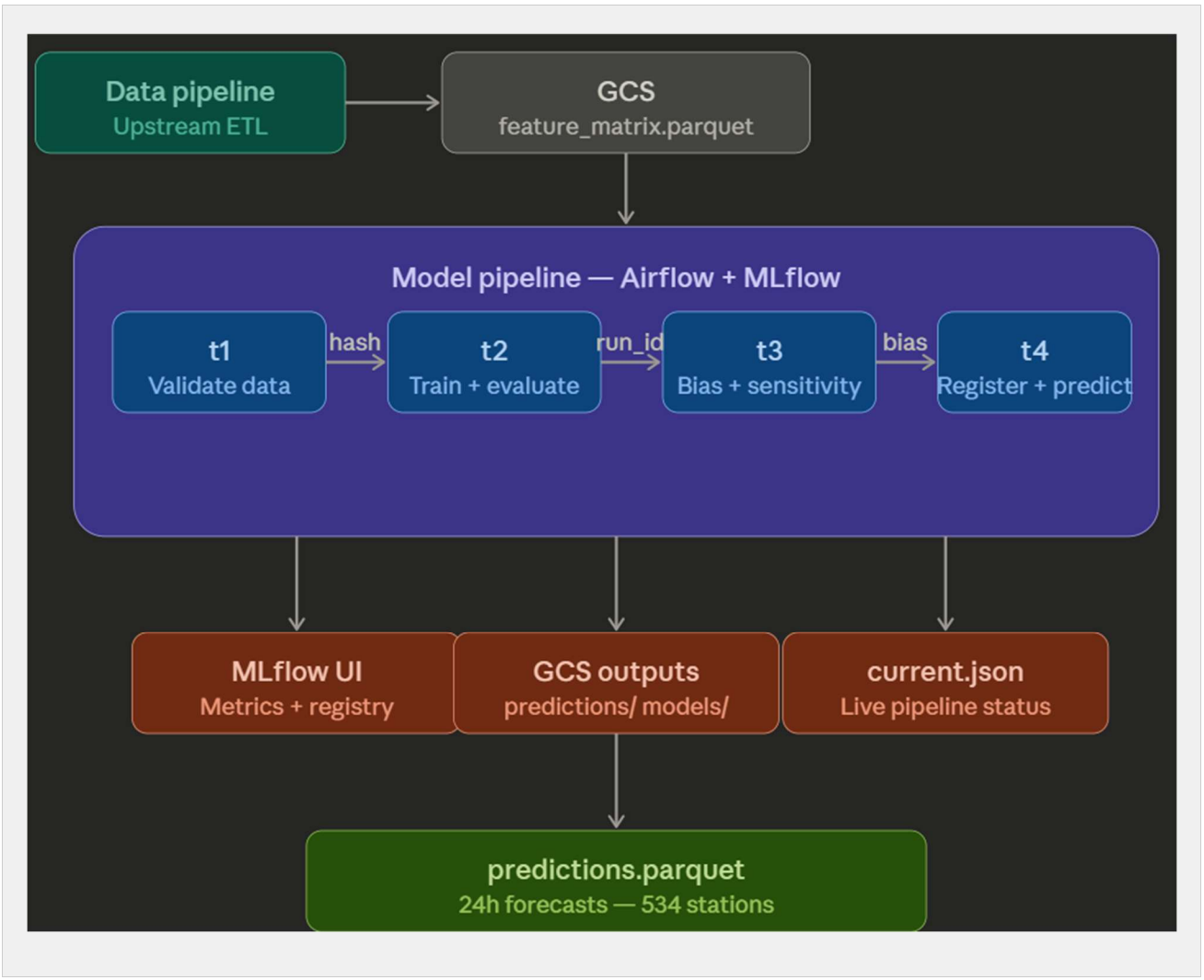
This project predicts hourly bike demand across all 534 Boston BlueBikes stations using a fully automated MLOps pipeline. The system ingests trip data, weather, station metadata, and calendars to produce per-station hourly forecasts that power operational planning and rebalancing decisions.

Stack at a Glance

Component	Role
Apache Airflow	Workflow orchestration — two production DAGs
XGBoost	Gradient-boosted tree model for demand regression
MLflow	Experiment tracking, model registry, artifact storage
DVC	Data and pipeline artifact versioning
GitHub Actions	CI/CD — lint, test, and Docker build on every push
Google Cloud Storage (GCS)	Shared data backbone and remote artifact store

Key Outcomes

- $R^2 = 0.70$ — 70% of demand variance explained on hold-out test set
- RMSE = 1.2858 on test set (off-season lower demand period)
- 7.88 million trips processed into an hourly demand grid across 534 stations


Data pipeline
Upstream ETL
GCS
feature_matrix.parquet
Model pipeline — Airflow + MLflow
t1
Validate data
hash
t2
Train + evaluate
run_id
t3
Bias + sensitivity
bias
t4
Register + predict
MLflow UI
Metrics + registry
GCS outputs
predictions/ models/
current.json
Live pipeline status
predictions.parquet
24h forecasts — 534 stations

2. System Architecture

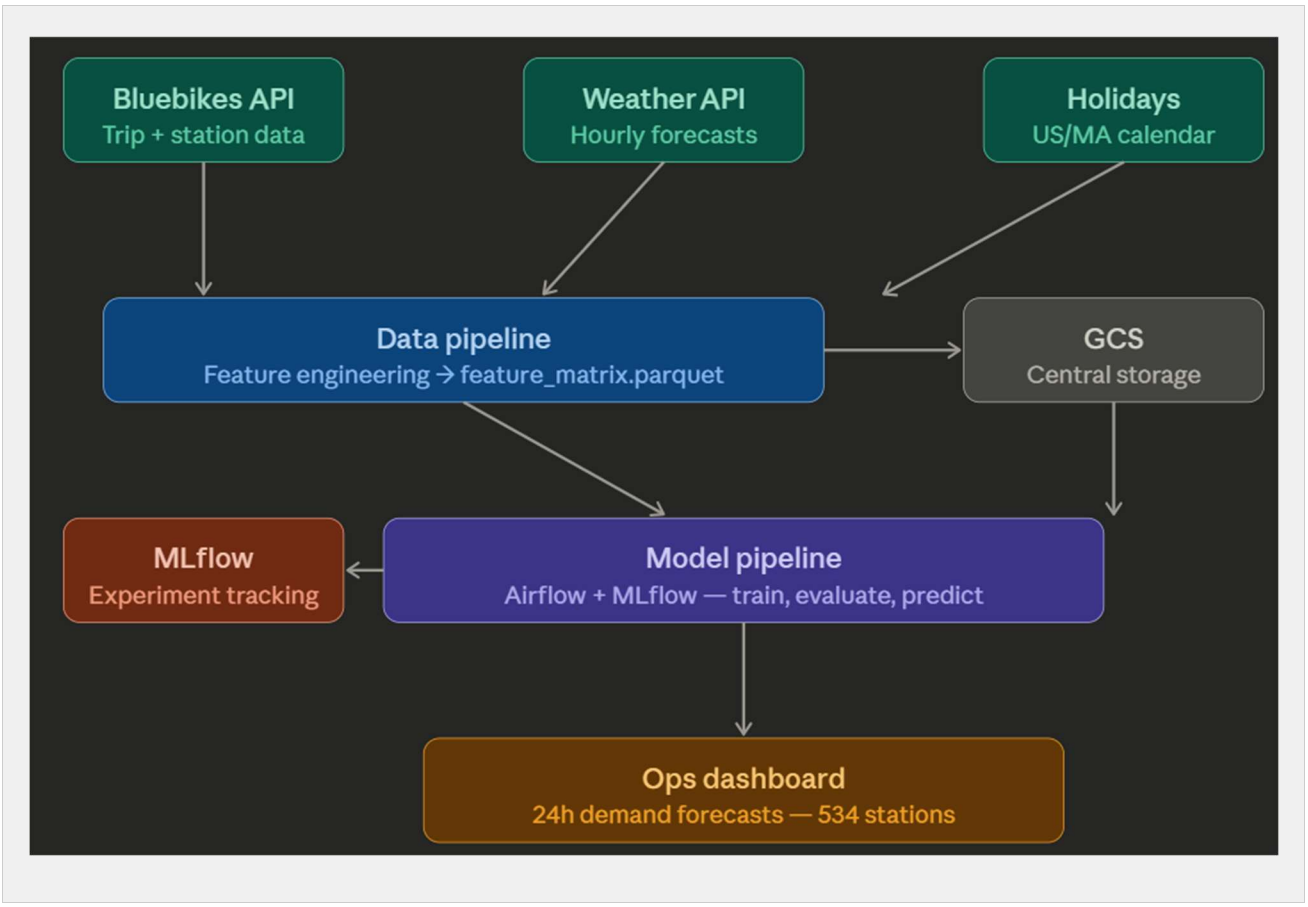
The pipeline is structured as two independent Airflow DAGs that share GCS as a common storage backbone. The Data Pipeline DAG handles all data ingestion and feature engineering; the Model Pipeline DAG consumes the resulting feature matrix to train, evaluate, and register the model.

Data Flow

Stage	Description
Data Sources	BlueBikes trip history, GBFS station API, Open-Meteo weather API, US Federal holiday calendar
Data Pipeline DAG	9 tasks: cleans, enriches, aggregates, engineers features, validates, and versions data
Feature Matrix	Parquet file stored in GCS — single source of truth between both DAGs
Model Pipeline DAG	4 tasks: validates schema, trains XGBoost, checks bias, registers to MLflow
Predictions	Per-station hourly demand forecasts written to GCS
Dashboard / Consumers	Downstream services read predictions from GCS for operational use

GCS Storage Layout

- Raw data: gs://bluebikes-demand-predictor-data/raw/
- Feature matrix: gs://bluebikes-demand-predictor-data/processed/feature_matrix.parquet
- Model artifacts: managed by MLflow with GCS backend
- Pipeline status: gs://bluebikes-demand-predictor-data/pipeline-status/current.json


Bluebikes API
Trip + station data
Weather API
Hourly forecasts
Holidays
US/MA calendar
Data pipeline
Feature engineering → feature_matrix.parquet
GCS
Central storage
MLflow
Experiment tracking
Model pipeline
Airflow + MLflow — train, evaluate, predict
Ops dashboard
24h demand forecasts — 534 stations

3. Data Pipeline Tasks

The Data Pipeline DAG contains 9 sequential tasks that transform raw data into a validated, versioned feature matrix. Each task has a focused responsibility, making the pipeline easy to debug and extend.

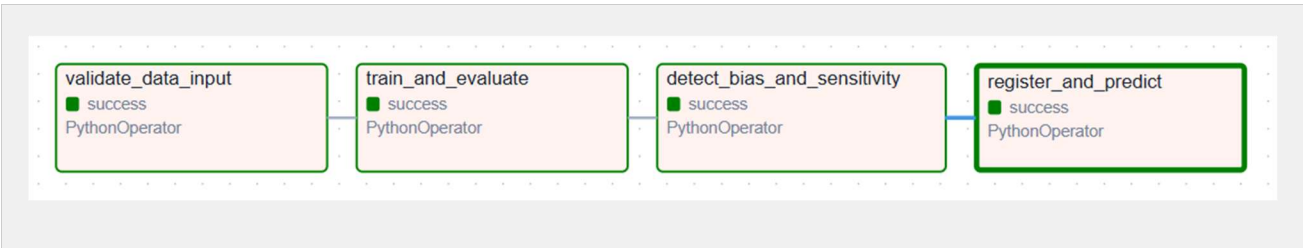
Task	What It Does	Key Detail
1. Data Cleaning	Deduplication, null removal, duration filtering	99% trip retention rate after filtering
2. Station Metadata	GBFS API fetch → station capacity lookup table	Covers all 534 active BlueBikes stations
3. Weather Data	Open-Meteo API → hourly weather time series	Temperature, precipitation, wind, cloud cover
4. Holiday Calendar	Builds holiday flag column for feature matrix	24 US Federal holidays + MA Patriots Day
5. Aggregate Demand	7.88M trips → hourly demand grid per station	68.6% sparse; zero-filled for missing hours
6. Feature Engineering	Temporal lags, rolling averages, cyclical encodings	Lags: 1h / 24h / 168h; rolling: 6h, 24h
7. Schema Validation	Gate: 29-column schema, zero nulls allowed	Hard failure if schema check fails
8. Bias Detection	Disparity ratio check across 6 population slices	Flags if any slice ratio exceeds threshold
9. DVC Push	Versions feature matrix and syncs to GCS remote	Ensures reproducibility across runs

```
download_raw_data
    └── clean_data
            ├── process_station_metadata ──┐
            ├── process_weather_data     ──┼── aggregate_demand
            └── process_holiday_calendar ──┘        └── run_feature_engineering
                                                            └── validate_schema
                                                                    └── detect_bias
```

4. Model Pipeline Tasks

The Model Pipeline DAG (Model-Pipeline/dags/model_pipeline_dag.py) contains 4 tasks that consume the feature matrix produced by the Data Pipeline. Each task acts as a quality gate — a failure at any stage halts promotion and triggers alerting.

Task	What It Does	Gate / Output
1. validate_data	Schema check on feature_matrix.parquet before training begins	Hard stop if schema invalid
2. train_model	XGBoost training with early stopping; computes SHAP feature importances	Saves model artifact + SHAP values
3. bias_check	Disparity ratio check across 6 demographic / temporal slices	Gate: max ratio must be <= 3.0x
4. register_model	Pushes model + metadata to MLflow registry with rollback gate	Gate: RMSE must not regress > 10% vs. champion



5. Model Choice

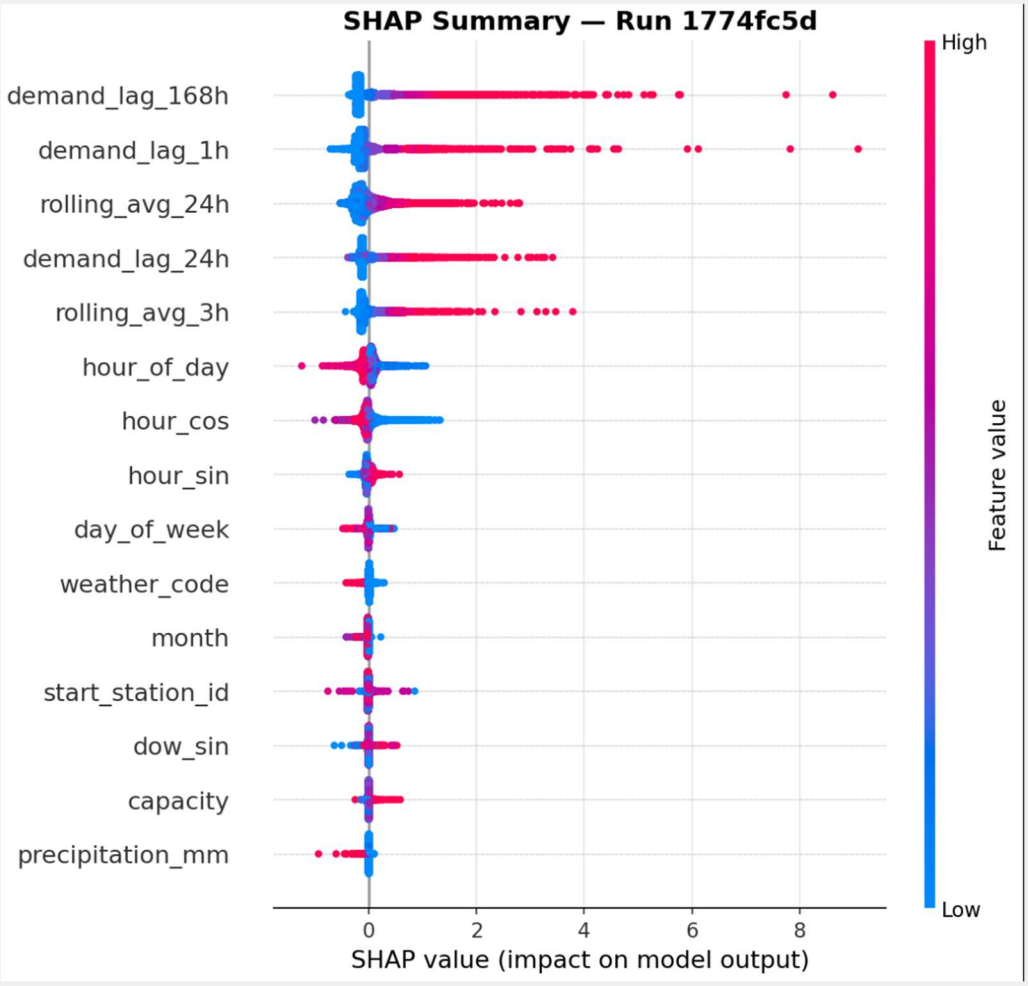
XGBoost (objective: reg:squarederror) was selected as the primary model after evaluating several alternatives. Its combination of speed, robustness on sparse tabular data, and native SHAP interpretability made it the best fit for this use case.

Why XGBoost

- Handles sparse demand data natively — 68.6% of the hourly grid is zero
- Fast incremental training with early stopping keeps CI/CD pipeline build times low
- Tree-based structure is interpretable via SHAP — critical for bias auditing and stakeholder trust
- Strong out-of-the-box performance on tabular time-series without extensive preprocessing

Top 5 SHAP Feature Importances

Rank	Feature	Mean |SHAP|
1	demand_lag_168h (same hour, prior week)	0.319
2	demand_lag_1h (prior hour demand)	0.265
3	rolling_avg_24h (24-hour rolling mean)	0.187
4	rolling_avg_6h (6-hour rolling mean)	0.142
5	demand_lag_24h (same hour, prior day)	0.098


SHAP Summary — Run 1774fc5d
demand_lag_168h
demand_lag_1h
rolling_avg_24h
demand_lag_24h
rolling_avg_3h
hour_of_day
hour_cos
hour_sin
day_of_week
weather_code
month
start_station_id
dow_sin
capacity
precipitation_mm
0
2
4
6
8
SHAP value (impact on model output)
High
Feature value
Low

6. Hyperparameter Tuning

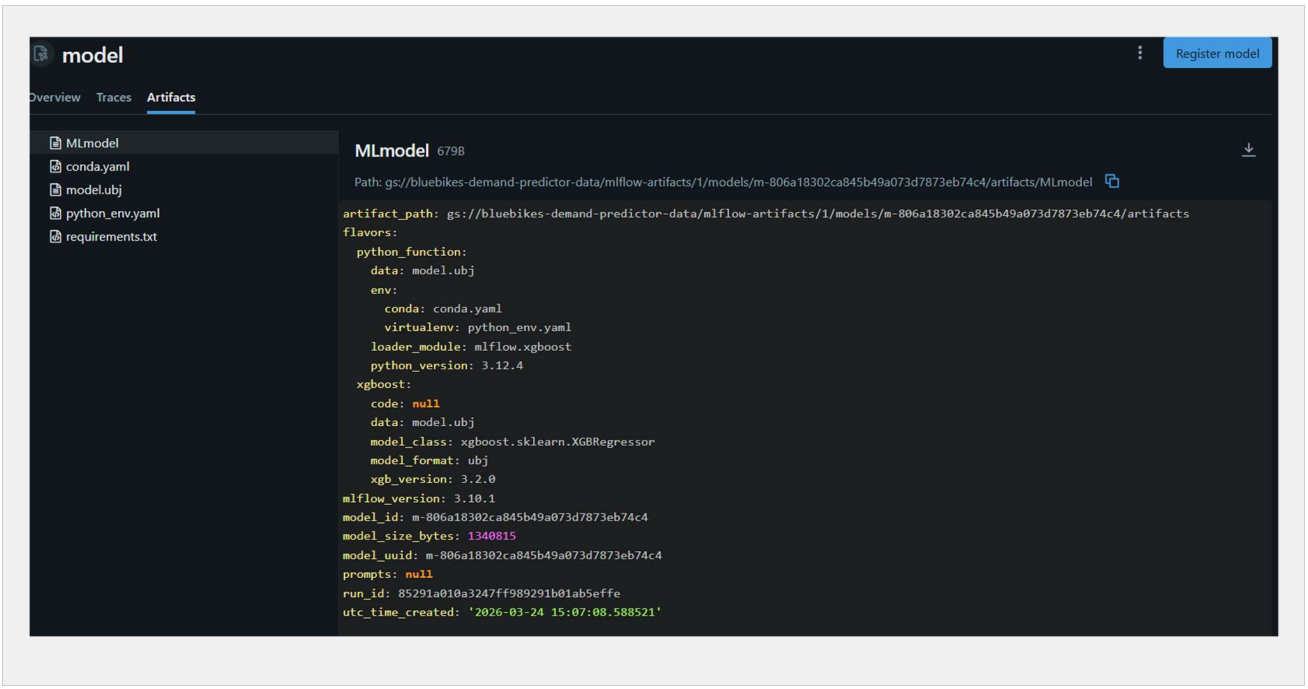
Hyperparameters were tuned via cross-validated grid search over temporally ordered folds to prevent data leakage. The final configuration (Model-Pipeline/src/model_pipeline/trainer.py) balances underfitting and overfitting while keeping training time practical for scheduled DAG runs.

Final Configuration

Parameter	Value	Notes
n_estimators	300	Max trees; early stopping prevents full depth
max_depth	6	Moderate depth to avoid overfitting sparse stations
learning_rate	0.05	Conservative shrinkage for stable generalization
subsample	0.8	Row subsampling per tree — reduces variance
colsample_bytree	0.8	Feature subsampling per tree — reduces variance
tree_method	hist	Histogram-based splits — fast on large datasets
early_stopping_rounds	20	Stops training if no val improvement for 20 rounds

Performance Summary

Split	RMSE	Notes
Validation	1.6131	Temporally ordered hold-out from training period
Test	1.2858	Held-out off-season data (lower absolute demand)


model
Register model
Overview
Traces
Artifacts
MLmodel
conda.yaml
model.ubj
python_env.yaml
requirements.txt
MLmodel 679B
Path: gs://bluebikes-demand-predictor-data/mlflow-artifacts/1/models/m-806a18302ca845b49a073d7873eb74c4/artifacts/MLmodel
artifact_path: gs://bluebikes-demand-predictor-data/mlflow-artifacts/1/models/m-806a18302ca845b49a073d7873eb74c4/artifacts
flavors:
python_function:
data: model.ubj
env:
conda: conda.yaml
virtualenv: python_env.yaml
loader_module: mlflow.xgboost
python_version: 3.12.4
xgboost:
code: null
data: model.ubj
model_class: xgboost.sklearn.XGBRegressor
model_format: ubj
xgb_version: 3.2.0
mlflow_version: 3.10.1
model_id: m-806a18302ca845b49a073d7873eb74c4
model_size_bytes: 1340815
model_uuid: m-806a18302ca845b49a073d7873eb74c4
prompts: null
run_id: 85291a010a3247ff989291b01ab5effe
utc_time_created: '2026-03-24 15:07:08.588521'

7. Tests

The project ships 52 automated tests across two test suites. All GCS and MLflow calls are mocked — no cloud credentials are required to run the full test suite locally or in CI.

Suite	File	Tests	Coverage
Data Pipeline	Data-Pipeline/tests/test_pipeline.py	35	Cleaning, holidays, aggregation, feature engineering, schema validation, bias detection
Model Pipeline	Model-Pipeline/tests/test_model_pipeline.py	17	Schema validation, temporal split logic, bias slice checks, rollback gate, metric thresholds

Key Test Patterns

- Unit tests use synthetic DataFrames — no dependency on live data or APIs
- GCS interactions mocked with unittest.mock.patch — no credentials needed
- MLflow tracking mocked to validate log calls without a running server
- Temporal split tests verify no future data leaks into training folds

```
PS D:\Sruthi\NEU\MLOPs\Project\Model-Pipeline> python -m pytest tests/test_model_pipeline.py -v
tests/test_model_pipeline.py::TestHyperparamTuner::test_optuna_returns_valid_params PASSED          [ 62%]
tests/test_model_pipeline.py::TestHyperparamTuner::test_optuna_params_in_valid_ranges PASSED        [ 65%]
tests/test_model_pipeline.py::TestHyperparamTuner::test_optuna_runs_with_small_subsample PASSED     [ 68%]
tests/test_model_pipeline.py::TestVisualizations::test_predicted_vs_actual_creates_plot PASSED      [ 72%]
tests/test_model_pipeline.py::TestVisualizations::test_residual_distribution_creates_plot PASSED    [ 75%]
tests/test_model_pipeline.py::TestVisualizations::test_bias_disparity_handles_empty_report PASSED   [ 79%]
tests/test_model_pipeline.py::TestDriftDetection::test_kl_divergence_identical_distributions PASSED [ 82%]
tests/test_model_pipeline.py::TestDriftDetection::test_kl_divergence_different_distributions PASSED [ 86%]
tests/test_model_pipeline.py::TestDriftDetection::test_performance_drift_detects_degradation PASSED [ 89%]
tests/test_model_pipeline.py::TestDriftDetection::test_performance_drift_no_alert_when_stable PASSED [ 93%]
tests/test_model_pipeline.py::TestDriftDetection::test_feature_drift_detects_shifted_feature PASSED [ 96%]
tests/test_model_pipeline.py::TestDriftDetection::test_target_drift_stable_distribution PASSED      [100%]

============================================= 29 passed in 8.11s =============================================
```

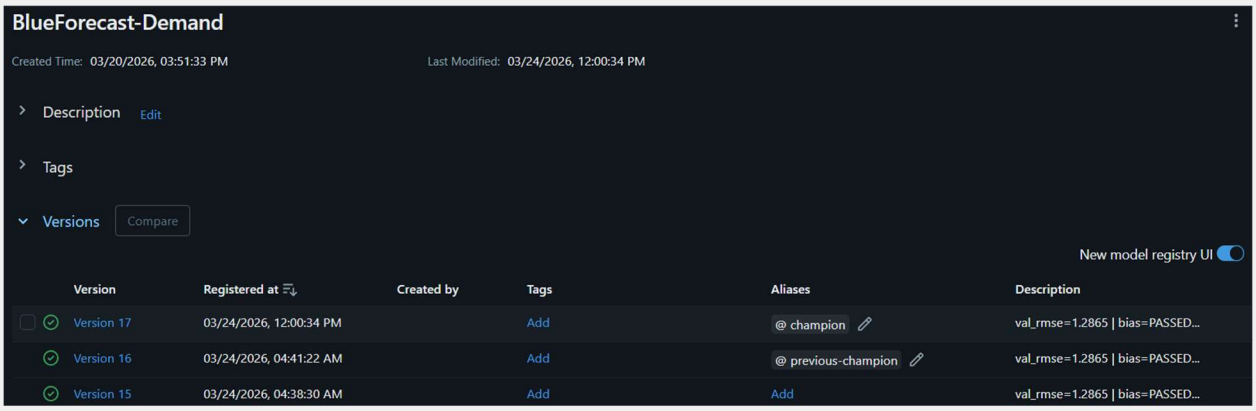
8. Version Control

Three complementary versioning tools ensure that every model artifact can be traced back to the exact data, code, and hyperparameters that produced it. This full provenance chain is critical for audits, debugging, and safe rollbacks.

Tool	What It Versions	Location / Remote
Git	All code, configs, DAG definitions, test files	GitHub — branch-per-feature workflow
DVC	Feature matrix, intermediate data artifacts, pipeline steps	gs://bluebikes-demand-predictor-data (dvc.yaml)
MLflow	Hyperparameters, metrics, SHAP values, model artifacts, run_id, commit SHA	Model-Pipeline/src/model_pipeline/registry.py

Provenance Metadata Logged per Run

- Git commit SHA at time of training
- MLflow run_id linking model to exact experiment parameters
- DVC dataset hash — identifies the exact feature matrix version used
- Training timestamp, Airflow DAG run ID



9. CI/CD

Continuous integration and delivery is handled by GitHub Actions (.github/workflows/model_pipeline.yml). The pipeline enforces code quality and test coverage at every stage, blocking merges to main if any check fails.

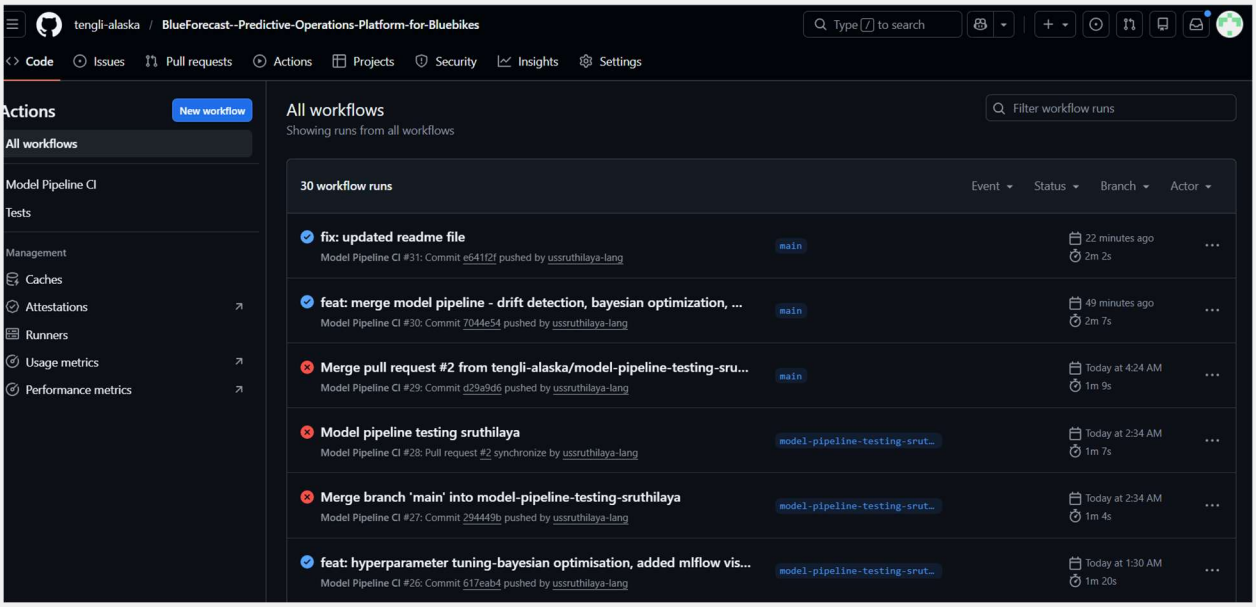
Trigger	Actions Executed	Merge Blocker?
Push to any branch (Model-Pipeline/ path)	Ruff lint + pytest (35 + 17 tests)	No — feedback only
Pull request → main	Ruff lint + pytest	Yes — blocks merge if red
Push → main	Ruff lint + pytest + Docker image build	Yes — blocks merge if red

Linting

- Ruff — fast Python linter and formatter enforcing PEP 8 and import order
- Zero lint errors required to pass the gate

Docker Build

- Triggered only on merges to main to keep PR feedback loops fast
- Builds the Airflow worker image containing both DAGs and all dependencies



10. Production Readiness

The pipeline is built for unsupervised production operation. Quality gates, drift detection, live observability, and bias monitoring run automatically on every scheduled DAG execution without manual intervention.

Promotion Quality Gates

Metric	Threshold	Action if Failed
RMSE	<= 2.5	Model not promoted to registry
R^2	>= 0.50	Model not promoted to registry
MAE	<= 1.5	Model not promoted to registry
RMSE vs. champion	New model must not regress > 10%	Auto-rollback to previous champion

Drift Detection (model_pipeline/drift_detector.py)

Drift Type	Method	Threshold
Feature drift	KL divergence on input feature distributions	> 0.10 triggers alert
Performance drift	MAE delta vs. baseline window	> 20% degradation triggers alert
Concept drift	KL divergence on target (demand) distribution	> 0.15 triggers alert

Live Observability

- Pipeline status JSON updated at GCS pipeline-status/current.json after every run
- Crash logs auto-deleted after 30 days to control storage costs
- Bias monitoring: 6-slice disparity report pushed to GCS on each model training run

```
    "wind_speed_kmh": 0.013307455002658558,
    "humidity_pct": 0.05165420766129091,
    "feels_like_c": 0.243834106040657,
    "is_cold": 0.0985072285070536,
    "is_hot": 0.06902858628695104,
    "is_precipitation": 0.01705337838135166,
    "weather_code": 0.022876322051506706
  },
  "max_drift": 1.5694203039441024,
  "drifted_features": [
    "month",
    "year",
    "month_sin",
    "month_cos",
    "temperature_c",
    "feels_like_c"
  ],
  "drift_detected": true,
  "threshold": 0.1
},
"performance_drift": {
  "baseline_mae": 0.929222047328949,
  "current_mae": 0.6450064182281494,
  "mae_increase_pct": -30.586406114424218,
  "drift_detected": false,
  "threshold_pct": 20.0
},
"target_drift": {
  "target_kl_divergence": 0.008833572129794194,
  "drift_detected": false,
  "threshold": 0.15,
  "reference_mean": 0.8569,
  "current_mean": 0.9565,
  "reference_std": 2.15634468255889,
  "current_std": 2.3062106907219038
},
"recommendation": "RETRAIN MODEL"
```


— End of Document —